

TP6, eleve : Bascule defensive (remediation)

- Jour 2 - Duree : ~1h00 - Dossier : `jour2-ctf/`
- Le livrable de ce TP compte pour 40 % de la note.

Contexte

Vous avez casse l'application toute la journee. Maintenant, on repare. Le flag seul ne suffit pas : c'est la capacite a remedier, et a raisonner aux trois niveaux, qui est evaluee.

Objectif

Choisir 3 vulnerabilites exploitees aujourd'hui, ecrire leur correction dans le code, verifier que l'attaque echoue, et remplir la matrice de remediation a 3 niveaux.

Deroule

1. Le code source est fourni dans `jour2-ctf/app/` (notamment `app.py` et les templates).
2. Pour chaque vulnerabilite choisie : - corrigez le code ; - relancez : `docker compose up -d --build` ; - re-tentez votre attaque du matin : elle doit desormais echouer.
3. Remplissez `docs/matrice-remediation-template.md` : pour chaque vulnerabilite, une contre-mesure au niveau Code, une au niveau Architecture, une au niveau Organisationnel.
4. Priorisez (probabilite x impact, effort de correction).

Les trois niveaux (rappel)

- **Code** : le correctif technique immediat (ex. requete parametree).
- **Architecture** : ce qui, dans la conception, reduit la classe entiere de failles (ex. compte DB a moindre privilege, autorisation centralisee).
- **Organisationnel** : ce qui empeche la faille de revenir (ex. SAST en CI, revue de code, formation, checklist de mise en prod).

Conseils

- Commencez par les vulnerabilites a fort impact (SQLi, IDOR, JWT).
- Testez chaque correction en rejouant l'attaque : une correction non verifiee ne compte pas.
- Pensez "classe de failles", pas seulement "ce cas precis".

Livrable attendu (note)

Matrice de remediation remplie pour 3 a 5 vulnerabilites + preuve que chaque attaque corrige e echoue (capture, `git diff`, ou description). C'est le coeur de l'evaluation.